

DEPLOYMENT ARCHITECTURE · REFERENCE DOCUMENT

Healthcare Clinical Intelligence System

One EC2 instance, one Docker container, one persistent EBS volume — provisioned by Terraform, built and pushed by GitHub Actions, managed entirely through SSM Session Manager with no SSH key and no open port 22. Secrets live in SSM Parameter Store, never in the image.

AWS EC2

EBS

ECR

SSM

IAM

Terraform

Docker

GitHub Actions

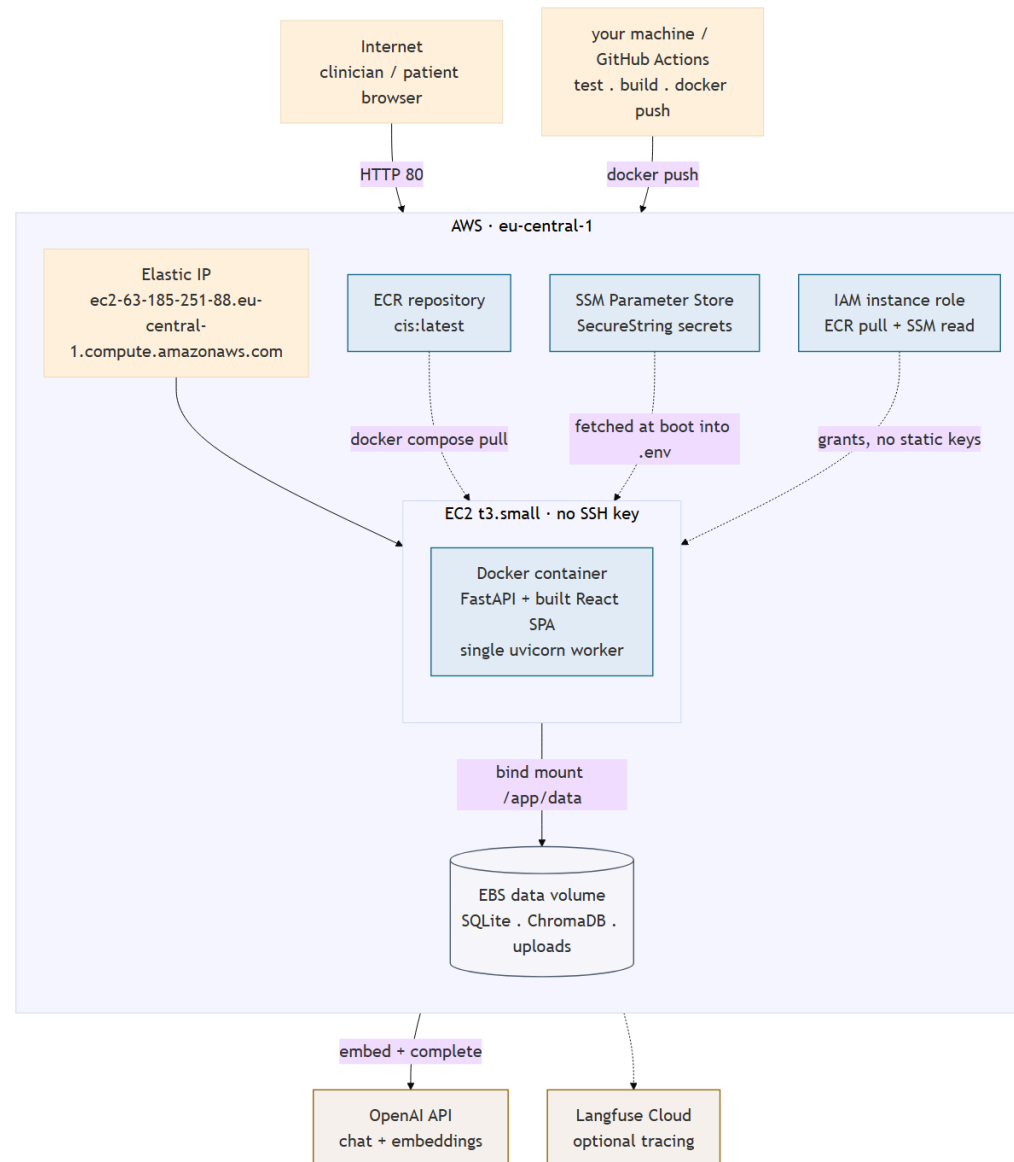
 LIVE — <http://ec2-63-185-251-88.eu-central-1.compute.amazonaws.com>

01 Why this shape

The app is SQLite + a local ChromaDB persist directory + local file uploads — single-writer, single-disk assumptions baked into `backend/config.py` from the start. That rules out anything that wants stateless or horizontally-scaled compute (Lambda, App Runner without a datastore migration, multiple ECS tasks behind a load balancer). It's a natural fit for one EC2 instance, one Docker container, one persistent EBS volume.

- Secrets (`OPENAI_API_KEY`, `SESSION_SECRET`, Langfuse keys) live in SSM Parameter Store as `SecureString`s — fetched by the instance at boot, never committed, never baked into the Docker image.
- No SSH key, no open port 22 by default — the instance is managed entirely through **SSM Session Manager** (`aws ssm start-session`), which only needs IAM, not a key pair or inbound rule.
- A single uvicorn worker, intentionally — SQLite is single-writer, so multiple workers against the same DB file risk "database is locked" errors under concurrent writes. Scale this vertically (a bigger instance type), not horizontally.
- The EBS data volume is declared separately from the instance's own lifecycle — replacing the instance (AMI update, instance-type change) never touches the volume or its data.

02 Deployment diagram



03 First-time setup

1. **Provision the infrastructure.** `terraform apply` from `deploy/terraform/` creates the ECR repository, EC2 instance + EBS data volume, security group, IAM role, placeholder SSM parameters, and an Elastic IP. The instance boots and runs `bootstrap.sh` as user-data — but there's no image in ECR yet, so the app itself isn't running after this step alone.
2. **Set the real secrets.** Terraform creates the SSM parameters as placeholders (`CHANGE_ME` / `UNSET` — SSM `SecureString`'s can't hold an empty string). Overwrite them with `aws ssm put-parameter --overwrite`.
3. **Build and push the first image** to ECR (`docker build --platform linux/amd64` , then `docker push`).
4. **Start the app** via `aws ssm send-command` running `docker compose pull && docker compose up -d` on the instance.
5. **Refresh secrets on the instance, if step 2 happened after first boot.** `bootstrap.sh` writes `/opt/app/.env` from SSM exactly once, at first boot. If real secrets are set in SSM afterward, re-fetch them into `.env` and `docker compose up -d --force-recreate` — updating SSM alone does not refresh a file already written to disk.

04 CI/CD & subsequent deploys

`.github/workflows/deploy.yml` runs on every push to `main` that touches `backend/`, `frontend/`, the `Dockerfile`, or `requirements.txt`:

1. **test** — installs `requirements.txt`, runs `pytest tests/ -q` (87 tests, no network/API key required).
2. **build-and-deploy** (only if tests pass) — logs in to ECR, builds and pushes the image tagged both `:latest` and `:${github.sha}`, then triggers a redeploy on the instance via `aws ssm send-command` running the same `docker compose pull && up -d` — no SSH key needed from CI either.

The IAM user behind the CI access keys is scoped to exactly `ecr:*` on the one repository and `ssm:SendCommand` / `ssm:GetCommandInvocation` on the one instance — never account-admin keys. `docker compose up -d` only recreates the container; the EBS-backed `/data/app` bind mount means the SQLite DB, ChromaDB, and uploads survive every redeploy.

05 Operations

Logs	<code>aws ssm start-session --target <instance-id></code> , then <code>docker logs -f \$(docker ps -q --filter ancestor=<ecr-repo-url>)</code> or <code>cat /var/log/bootstrap.log</code> for the first-boot script.
Backups	Everything that matters lives on the EBS data volume (<code>/data/app</code> on the host). Snapshot it with <code>aws ec2 create-snapshot --volume-id <id></code> .
Health check	<code>GET /healthz</code> — executes <code>SELECT 1</code> against SQLite, not just "process is up." An unmounted data volume fails this instead of lying about liveness.
Scaling	Don't run more than one container against the same SQLite file. Resize the instance type if needed, rather than adding more of them.
HTTPS	Not yet enabled — the default setup serves plain HTTP. A Caddy reverse-proxy container in front of the app is the documented path once a real domain is pointed at the instance (see <code>DEPLOYMENT.md</code>).
Tearing down	<code>terraform destroy</code> deletes the EC2 instance, security group, IAM role, ECR repo, and the EBS data volume — snapshot first if the data matters.

06 This deployment

URL	<code>http://ec2-63-185-251-88.eu-central-1.compute.amazonaws.com</code>
Region	eu-central-1 (Frankfurt)
Instance	t3.small, no SSH key — SSM Session Manager only
ECR repository	<code>509124060793.dkr.ecr.eu-central-1.amazonaws.com/cis</code>
Data volume	20 GB EBS gp3, bind-mounted at <code>/data/app</code>
Seed data	2 doctors, 2 radiologists, 5 patients, 12 reports run through the full 5-agent pipeline, 344 reference documents embedded — password <code>password123</code> for every seeded account

Known open items

- HTTPS not yet configured — plain HTTP on the AWS-assigned hostname; needs a custom domain to add TLS via Caddy.
- AWS credentials currently in use for admin operations are the account's **root** access key, not a scoped IAM user — worth narrowing before further changes.
- No custom/branded domain — the canonical URL is the free, AWS-assigned public DNS hostname, which has the instance's IP address embedded in it by design.